



Online Payment Integration Guide

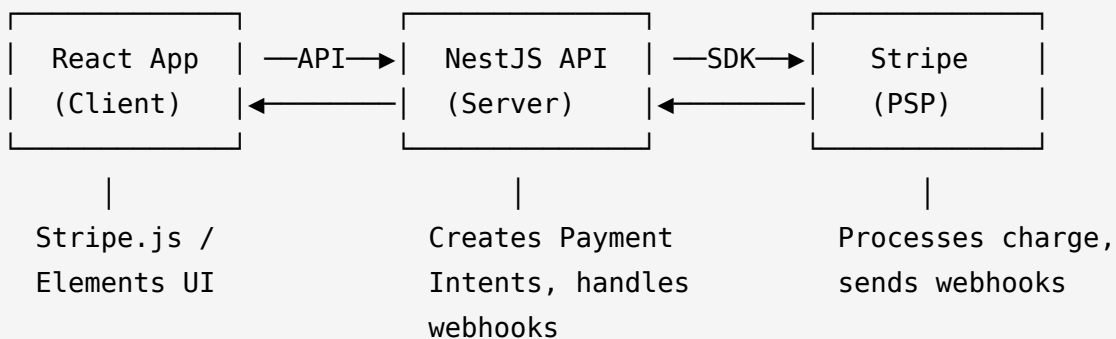
Stack: React (Frontend) · NestJS (Backend) · Stripe (Payment Provider) · TypeScript

Accepted Methods: Visa Cards · Google Pay · Apple Pay · PayPal

Table of Contents

1. [Architecture Overview](#)
2. [Prerequisites](#)
3. [Stripe Dashboard Setup](#)
4. [Backend Setup \(NestJS\)](#)
5. [Frontend Setup \(React\)](#)
6. [Implementing Card Payments \(Visa\)](#)
7. [Adding Google Pay](#)
8. [Adding Apple Pay](#)
9. [Adding PayPal](#)
10. [Webhook Handling](#)
11. [Testing](#)
12. [Going to Production](#)

1. Architecture Overview



Flow Summary:

1. User picks a payment method on the React frontend.
 2. Frontend calls the NestJS API to create a **PaymentIntent**.
 3. NestJS uses the Stripe Node SDK to create the intent and returns the `client_secret`.
 4. Frontend uses `@stripe/react-stripe-js` to confirm the payment with the chosen method.
 5. Stripe processes the payment and sends a **webhook** to NestJS to confirm its status.
-

2. Prerequisites

Tool	Minimum Version
Node.js	18+
npm / yarn / pnpm	latest
TypeScript	5+
Stripe Account	dashboard.stripe.com
PayPal Business Account	developer.paypal.com

Required API Keys (from Stripe Dashboard)

- `STRIPE_PUBLISHABLE_KEY` — exposed to the frontend
 - `STRIPE_SECRET_KEY` — backend only, **never** expose to the client
 - `STRIPE_WEBHOOK_SECRET` — for verifying webhook signatures
-

3. Stripe Dashboard Setup

3.1 Enable Payment Methods

1. Log in to [Stripe Dashboard](#).
2. Go to **Settings** → **Payment Methods**.
3. Enable the following:
 - ☒ **Cards** (Visa, Mastercard, etc.) — enabled by default
 - ☒ **Google Pay** — automatically available when Cards are enabled
 - ☒ **Apple Pay** — requires domain verification (see [Step 8](#))
 - ☒ **PayPal** — enable under Payment Methods (available in supported regions)

3.2 Get API Keys

1. Go to **Developers** → **API keys**.
2. Copy the **Publishable key** and **Secret key** (use **test mode** keys during development).

3.3 Create a Webhook Endpoint

1. Go to **Developers** → **Webhooks** → **Add endpoint**.
 2. Set URL to `https://your-api.com/payments/webhook`.
 3. Select events:
 - `payment_intent.succeeded`
 - `payment_intent.payment_failed`
 - `payment_intent.canceled`
 4. Copy the **Signing secret** for `STRIPE_WEBHOOK_SECRET`.
-

4. Backend Setup (NestJS)

4.1 Create the NestJS Project

```
npx -y @nestjs/cli new payment-api --strict --skip-git --package-manager npm  
cd payment-api
```

4.2 Install Dependencies

```
npm install stripe  
npm install -D @types/express
```

4.3 Environment Variables

Create `.env` in the project root:

```
STRIPE_SECRET_KEY=sk_test_...  
STRIPE_WEBHOOK_SECRET=whsec_...  
STRIPE_PUBLISHABLE_KEY=pk_test_...  
FRONTEND_URL=http://localhost:5173
```

Install and configure `@nestjs/config`:

```
npm install @nestjs/config
```

`src/app.module.ts`

```
import { Module } from "@nestjs/common";
import { ConfigModule } from "@nestjs/config";
import { PaymentsModule } from "../payments/payments.module";

@Module({
  imports: [ConfigModule.forRoot({ isGlobal: true }), PaymentsModule],
})
export class AppModule {}
```

4.4 Create the Payments Module

```
nest generate module payments
nest generate controller payments
nest generate service payments
```

4.5 Stripe Service

```
src/payments/payments.service.ts
```

```

import { Injectable } from "@nestjs/common";
import { ConfigService } from "@nestjs/config";
import Stripe from "stripe";

@Injectable()
export class PaymentsService {
  private stripe: Stripe;

  constructor(private configService: ConfigService) {
    this.stripe = new Stripe(
      this.configService.get<string>("STRIPE_SECRET_KEY")!,
      { apiVersion: "2024-12-18.acacia" },
    );
  }

  /**
   * Creates a PaymentIntent that supports cards, Google Pay,
   * Apple Pay, and PayPal.
   */
  async createPaymentIntent(
    amount: number, // in cents (e.g., 2000 = $20.00)
    currency: string = "usd",
    metadata?: Record<string, string>,
  ): Promise<Stripe.PaymentIntent> {
    return this.stripe.paymentIntents.create({
      amount,
      currency,
      payment_method_types: [
        "card", // Visa, Mastercard, etc. – also enables Google Pay & Apple Pay
        "paypal", // PayPal
      ],
      metadata: metadata ?? {},
    });
  }

  /**
   * Retrieves a PaymentIntent to check its current status.
   */
  async getPaymentIntent(id: string): Promise<Stripe.PaymentIntent> {

```

```
        return this.stripe.paymentIntents.retrieve(id);
    }

    /**
     * Constructs and verifies a Stripe webhook event.
     */
    constructWebhookEvent(payload: Buffer, signature: string): Stripe.Event {
        const secret = this.configService.get<string>("STRIPE_WEBHOOK_SECRET")!;
        return this.stripe.webhooks.constructEvent(payload, signature, secret);
    }
}
```

4.6 Payments Controller

src/payments/payments.controller.ts

```

import {
  Controller,
  Post,
  Body,
  Req,
  Res,
  Headers,
  HttpStatus,
  RawBodyRequest,
} from "@nestjs/common";
import { Request, Response } from "express";
import { PaymentsService } from "../payments.service";

// ——— DTOs ———

interface CreatePaymentIntentDto {
  amount: number; // amount in cents
  currency?: string; // default: 'usd'
  metadata?: Record<string, string>;
}

interface PaymentIntentResponse {
  clientSecret: string;
  paymentIntentId: string;
}

// ——— Controller ———

@Controller("payments")
export class PaymentsController {
  constructor(private readonly paymentsService: PaymentsService) {}

  /**
   * POST /payments/create-intent
   * Creates a new PaymentIntent and returns the client_secret.
   */
  @Post("create-intent")
  async createPaymentIntent(
    @Body() dto: CreatePaymentIntentDto,
  )

```

```

): Promise<PaymentIntentResponse> {
    const intent = await this.paymentsService.createPaymentIntent(
        dto.amount,
        dto.currency,
        dto.metadata,
    );

    return {
        clientSecret: intent.client_secret!,
        paymentIntentId: intent.id,
    };
}

/**
 * POST /payments/webhook
 * Handles incoming Stripe webhook events.
 * IMPORTANT: This must receive the raw body (not parsed JSON).
 */
@Post("webhook")
async handleWebhook(
    @Req() req: RawBodyRequest<Request>,
    @Res() res: Response,
    @Headers("stripe-signature") signature: string,
): Promise<void> {
    let event;

    try {
        event = this.paymentsService.constructWebhookEvent(
            req.rawBody!,
            signature,
        );
    } catch (err) {
        console.error("⚠ Webhook signature verification failed:", err);
        res.status(HttpStatus.BAD_REQUEST).send(`Webhook Error: ${err}`);
        return;
    }

    // — Handle the event —
    switch (event.type) {

```

```
case "payment_intent.succeeded": {
  const paymentIntent = event.data.object;
  console.log(`✅ Payment succeeded: ${paymentIntent.id}`);
  // TODO: Fulfill the order, update DB, send confirmation email, etc.
  break;
}
case "payment_intent.payment_failed": {
  const paymentIntent = event.data.object;
  console.log(`❌ Payment failed: ${paymentIntent.id}`);
  // TODO: Notify the user, log the failure, etc.
  break;
}
default:
  console.log(`❗ Unhandled event type: ${event.type}`);
}

res.status(HttpStatus.OK).json({ received: true });
}
```

4.7 Enable Raw Body Parsing for Webhooks

src/main.ts

```
import { NestFactory } from "@nestjs/core";
import { AppModule } from "../app.module";

async function bootstrap() {
  const app = await NestFactory.create(AppModule, {
    rawBody: true, // ← required for Stripe webhook signature verification
  });

  app.enableCors({
    origin: process.env.FRONTEND_URL || "http://localhost:5173",
  });

  await app.listen(3000);
  console.log("🚀 Payment API running on http://localhost:3000");
}
bootstrap();
```

4.8 Payments Module

src/payments/payments.module.ts

```
import { Module } from "@nestjs/common";
import { PaymentsController } from "../payments.controller";
import { PaymentsService } from "../payments.service";

@Module({
  controllers: [PaymentsController],
  providers: [PaymentsService],
  exports: [PaymentsService],
})
export class PaymentsModule {}
```

5. Frontend Setup (React)

5.1 Create the React App (Vite + TypeScript)

```
npx -y create-vite@latest payment-client --template react-ts  
cd payment-client
```

5.2 Install Stripe Dependencies

```
npm install @stripe/stripe-js @stripe/react-stripe-js
```

5.3 Environment Variables

Create `.env` in the React project root:

```
VITE_STRIPE_PUBLISHABLE_KEY=pk_test_...  
VITE_API_URL=http://localhost:3000
```

5.4 Initialize Stripe

`src/lib/stripe.ts`

```
import { loadStripe } from "@stripe/stripe-js";  
  
export const stripePromise = loadStripe(  
  import.meta.env.VITE_STRIPE_PUBLISHABLE_KEY,  
);
```

5.5 API Helper

`src/lib/api.ts`

```
const API_URL = import.meta.env.VITE_API_URL;

interface CreateIntentPayload {
  amount: number;
  currency?: string;
  metadata?: Record<string, string>;
}

interface CreateIntentResponse {
  clientSecret: string;
  paymentIntentId: string;
}

export async function createPaymentIntent(
  payload: CreateIntentPayload,
): Promise<CreateIntentResponse> {
  const res = await fetch(`${API_URL}/payments/create-intent`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(payload),
  });

  if (!res.ok) {
    throw new Error("Failed to create payment intent");
  }

  return res.json();
}
```

6. Implementing Card Payments (Visa)

This is the primary payment method. Google Pay and Apple Pay piggyback on the `card` payment method type — Stripe's **Payment Element** renders them automatically when detected.

6.1 Checkout Page Component

`src/components/CheckoutPage.tsx`

```

import React, { useState } from 'react';
import { Elements } from '@stripe/react-stripe-js';
import { stripePromise } from '../lib/stripe';
import { createPaymentIntent } from '../lib/api';
import { CheckoutForm } from './CheckoutForm';

export const CheckoutPage: React.FC = () => {
  const [clientSecret, setClientSecret] = useState<string | null>(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  const handleCheckout = async () => {
    setLoading(true);
    setError(null);

    try {
      const { clientSecret } = await createPaymentIntent({
        amount: 2000, // $20.00
        currency: 'usd',
        metadata: { orderId: 'order_123' },
      });
      setClientSecret(clientSecret);
    } catch (err) {
      setError('Failed to initialize payment. Please try again.');
```

```

    } finally {
      setLoading(false);
    }
  };

  if (!clientSecret) {
    return (
      <div className="checkout-page">
        <h1>Checkout</h1>
        <p>Total: <strong>$20.00</strong></p>
        {error && <p className="error">{error}</p>}
        <button onClick={handleCheckout} disabled={loading}>
          {loading ? 'Loading...' : 'Proceed to Payment'}
        </button>
      </div>
    );
  }

```

```

    );
  }

  return (
    <Elements
      stripe={stripePromise}
      options={{
        clientSecret,
        appearance: {
          theme: 'stripe',          // or 'night', 'flat'
          variables: {
            colorPrimary: '#0570de',
            borderRadius: '8px',
          },
        },
      }}
    >
      <CheckoutForm />
    </Elements>
  );
};

```

6.2 Checkout Form Component (Payment Element)

src/components/CheckoutForm.tsx

```

import React, { useState, FormEvent } from 'react';
import {
  PaymentElement,
  useStripe,
  useElements,
} from '@stripe/react-stripe-js';

export const CheckoutForm: React.FC = () => {
  const stripe = useStripe();
  const elements = useElements();

  const [message, setMessage] = useState<string | null>(null);
  const [isProcessing, setIsProcessing] = useState(false);

  const handleSubmit = async (e: FormEvent) => {
    e.preventDefault();

    if (!stripe || !elements) return;

    setIsProcessing(true);

    const { error } = await stripe.confirmPayment({
      elements,
      confirmParams: {
        return_url: `${window.location.origin}/payment-success`,
      },
    });

    if (error) {
      setMessage(error.message ?? 'An unexpected error occurred.');
    }
    // If no error, the user is redirected to `return_url`.

    setIsProcessing(false);
  };

  return (
    <form onSubmit={handleSubmit} className="checkout-form">
      {/*

```

The PaymentElement automatically renders:

- Card input (Visa, Mastercard, etc.)
- Google Pay button (if available on device/browser)
- Apple Pay button (if available on device/browser)
- PayPal button (if enabled in Stripe dashboard)

```
*/}
```

```
<PaymentElement />
```

```
<button disabled={isProcessing || !stripe || !elements}>
```

```
  {isProcessing ? 'Processing...' : 'Pay Now'}
```

```
</button>
```

```
  {message && <div className="payment-message">{message}</div>}
```

```
</form>
```

```
);
```

```
};
```

6.3 Payment Success Page

src/components/PaymentSuccess.tsx

```

import React, { useEffect, useState } from 'react';
import { useSearchParams } from 'react-router-dom';
import { stripePromise } from '../lib/stripe';

export const PaymentSuccess: React.FC = () => {
  const [searchParams] = useSearchParams();
  const [status, setStatus] = useState<string>('loading');

  useEffect(() => {
    const clientSecret = searchParams.get('payment_intent_client_secret');
    if (!clientSecret) {
      setStatus('error');
      return;
    }

    stripePromise.then((stripe) => {
      if (!stripe) return;

      stripe.retrievePaymentIntent(clientSecret).then(({ paymentIntent }) => {
        switch (paymentIntent?.status) {
          case 'succeeded':
            setStatus('success');
            break;
          case 'processing':
            setStatus('processing');
            break;
          default:
            setStatus('error');
        }
      });
    });
  }, [searchParams]);

  return (
    <div className="payment-result">
      {status === 'loading' && <p>Verifying payment...</p>}
      {status === 'success' && <h1>✅ Payment Successful!</h1>}
      {status === 'processing' && <h1>⌚ Payment is processing...</h1>}
      {status === 'error' && <h1>❌ Payment failed. Please try again.</h1>}
    </div>
  );
};

```

```
    </div>
  );
};
```

7. Adding Google Pay

No additional code is needed. Google Pay is automatically available through Stripe's `PaymentElement` when:

1. ☒ **Cards** are enabled in Stripe Dashboard (they are by default).
2. ☒ The user's browser supports Google Pay (Chrome on Android/Desktop with a saved card).
3. ☒ You're using the `PaymentElement` component (already done in Step 6).

Stripe detects the browser's Google Pay capability and shows the Google Pay button automatically.

Verification

- Open your app in **Chrome** with a Google account that has a saved payment method.
- The Google Pay button should appear above the card input.

8. Adding Apple Pay

Apple Pay requires **domain verification**, but no additional frontend code.

8.1 Domain Verification

1. In Stripe Dashboard, go to **Settings** → **Payment Methods** → **Apple Pay**.
2. Click **Add new domain**.
3. Enter your domain (e.g., `yoursite.com`).
4. Download the **domain verification file**.

5. Host it at:

```
https://yoursite.com/.well-known/apple-developer-merchantid-domain-association
```

Serving the Verification File in NestJS

`src/main.ts` — Add static file serving:

```
import { NestFactory } from "@nestjs/core";
import { AppModule } from "./app.module";
import { join } from "path";
import { NestExpressApplication } from "@nestjs/platform-express";

async function bootstrap() {
  const app = await NestFactory.create<NestExpressApplication>(AppModule, {
    rawBody: true,
  });

  // Serve the Apple Pay domain verification file
  app.useStaticAssets(join(__dirname, "..", "public"), {
    prefix: "/.well-known/",
  });

  app.enableCors({
    origin: process.env.FRONTEND_URL || "http://localhost:5173",
  });

  await app.listen(3000);
}
bootstrap();
```

Place the file at:

```
public/.well-known/apple-developer-merchantid-domain-association
```

8.2 How It Works

Once verified, Apple Pay appears automatically in the `PaymentElement` on:

- **Safari** on macOS (with Touch ID or Apple Pay configured)
 - **Safari** on iOS (with Apple Pay configured)
-

9. Adding PayPal

9.1 Enable PayPal in Stripe Dashboard

1. Go to **Settings** → **Payment Methods**.
2. Find **PayPal** and click **Turn on**.
3. Connect your PayPal Business account when prompted.

9.2 Backend — Already Handled

In `payments.service.ts`, `payment_method_types` already includes `'paypal'`:

```
payment_method_types: [  
  'card',    // Cards + Google Pay + Apple Pay  
  'paypal',  // PayPal  
],
```

9.3 Frontend — Already Handled

The `PaymentElement` component automatically renders a **PayPal button** when:

1. ☒ PayPal is enabled in Stripe Dashboard.
2. ☒ `'paypal'` is in the `payment_method_types` of the `PaymentIntent`.
3. ☒ The currency is supported by PayPal (e.g., `usd`, `eur`, `gbp`).

9.4 PayPal-Specific Redirect Handling

PayPal payments use a redirect flow. Update `confirmPayment` to handle it:

```
const { error } = await stripe.confirmPayment({
  elements,
  confirmParams: {
    return_url: `${window.location.origin}/payment-success`,
    // PayPal requires a return_url – already provided above
  },
});
```

This is already done in the `CheckoutForm` component from Step 6.

10. Webhook Handling

Webhooks are your **source of truth** for payment status. Never rely solely on the frontend redirect.

10.1 Events to Handle

Event	Description
<code>payment_intent.succeeded</code>	Payment completed successfully
<code>payment_intent.payment_failed</code>	Payment attempt failed
<code>payment_intent.canceled</code>	Payment was canceled
<code>charge.refunded</code>	A refund was issued

10.2 Best Practices

```
// In your webhook handler, always:

// 1. Verify the signature (already done in PaymentsService)
// 2. Use idempotency – check if you've already processed this event
// 3. Respond with 200 quickly, then process async

case 'payment_intent.succeeded': {
  const paymentIntent = event.data.object as Stripe.PaymentIntent;

  // Check idempotency
  const alreadyProcessed = await this.ordersService.isProcessed(
    paymentIntent.id,
  );
  if (alreadyProcessed) break;

  // Fulfill the order
  await this.ordersService.fulfill(paymentIntent.metadata.orderId, {
    stripePaymentId: paymentIntent.id,
    amount: paymentIntent.amount,
    currency: paymentIntent.currency,
    paymentMethod: paymentIntent.payment_method_types[0],
  });

  break;
}
```

10.3 Local Webhook Testing

Use the **Stripe CLI** to forward webhooks to your local server:

```
# Install the Stripe CLI
# macOS: brew install stripe/stripe-cli/stripe
# Linux: see https://stripe.com/docs/stripe-cli

# Login
stripe login

# Forward webhooks to your local NestJS server
stripe listen --forward-to localhost:3000/payments/webhook
```

The CLI will output a webhook signing secret (`whsec_...`) — use it as `STRIPE_WEBHOOK_SECRET` in your `.env` .

11. Testing

11.1 Test Card Numbers

Card Number	Scenario
4242 4242 4242 4242	✅ Successful Visa payment
4000 0025 0000 3155	🔒 Requires 3D Secure authentication
4000 0000 0000 9995	❌ Declined (insufficient funds)
4000 0000 0000 0077	❌ Declined (always fails)

Use any future expiry date (e.g., 12/34) and any 3-digit CVC.

11.2 Test PayPal

In test mode, Stripe provides a simulated PayPal flow — no real PayPal account needed.

11.3 Test Google Pay

- Use Chrome with a saved test card.

- Works in Stripe test mode with test cards added to Google Pay.

11.4 Test Apple Pay

- Test on Safari with Apple Pay configured.
- Use Stripe test mode; no real charges are made.

11.5 Run Both Services

```
# Terminal 1 – Backend
cd payment-api
npm run start:dev

# Terminal 2 – Frontend
cd payment-client
npm run dev

# Terminal 3 – Stripe Webhooks (optional, for local testing)
stripe listen --forward-to localhost:3000/payments/webhook
```

12. Going to Production

12.1 Checklist

- ☐ Switch from **test** API keys to **live** API keys in Stripe Dashboard.
- ☐ Update `.env` files with live keys on your server.
- ☐ Verify Apple Pay domain for your production domain.
- ☐ Set up a production webhook endpoint in Stripe Dashboard.
- ☐ Enable HTTPS on both frontend and backend.
- ☐ Implement proper error handling and logging.
- ☐ Add rate limiting to the `/payments/create-intent` endpoint.
- ☐ Store payment records in your database.
- ☐ Set up monitoring/alerts for failed payments.
- ☐ Test the full payment flow with real (small amount) transactions.

12.2 Security Reminders

Rule	Detail
Never expose the Secret Key	<code>STRIPE_SECRET_KEY</code> must only exist on the server
Always verify webhooks	Use <code>constructEvent</code> to verify the <code>stripe-signature</code> header
Use HTTPS	Stripe requires HTTPS in production
Validate amounts server-side	Never trust amounts from the client
PCI compliance	Using Stripe Elements means your server never handles raw card data (SAQ-A)

Summary of Payment Method Visibility

Method	How It Appears	Requirements
Visa / Cards	Card input fields in PaymentElement	None (default)
Google Pay	Button above card fields	Chrome + saved card
Apple Pay	Button above card fields	Safari + domain verification
PayPal	Button in PaymentElement	Enabled in Stripe Dashboard + connected PayPal Business account

File Structure (Final)

```
payment-api/                                # NestJS Backend
├── src/
│   ├── app.module.ts
│   ├── main.ts
│   └── payments/
│       ├── payments.module.ts
│       ├── payments.controller.ts
│       └── payments.service.ts
├── public/
│   └── .well-known/
│       └── apple-developer-merchantid-domain-association
├── .env
└── package.json
```

```
payment-client/                             # React Frontend
├── src/
│   ├── lib/
│   │   ├── stripe.ts
│   │   └── api.ts
│   ├── components/
│   │   ├── CheckoutPage.tsx
│   │   ├── CheckoutForm.tsx
│   │   └── PaymentSuccess.tsx
│   ├── App.tsx
│   └── main.tsx
├── .env
└── package.json
```